MEMECOIN CLASH

Simulation multi-agents polymorphe - Rapport de projet

4 factions. 1 arène. 24 messages.

Doge, Shiba, Pepe et Bonk explorent une carte, échangent leurs connaissances et livrent les messages collectés à leur Maître.



Plateau Swing rendu à partir du moteur, au tour 6. Les logos identifient les factions ; M désigne un Maître.

Équipe et référent

Identités des membres à renseigner.
Référent : À renseigner

Dossier remis

Sources Java, exécutable JAR avec logos intégrés, lanceurs Windows et Unix, tests autonomes, trois diagrammes UML et ce rapport de six pages.

Version à compléter : identité de l'équipe et contributions réelles.

Architecture et concepts objet

Responsabilités, héritage, encapsulation et polymorphisme

Un moteur indépendant de la fenêtre

Simulation conserve les agents, les Maîtres et le journal, puis applique les règles. **GameMap** gère coordonnées, obstacles, zones sûres, voisins et direction du retour. **GameSettings** porte une configuration immuable. **GameWindow**, **BoardPanel** et **SettingsDialog** pilotent, affichent et configurent la partie.

Héritage : factoriser données et déplacement

Agent rassemble équipe, position, énergie, dernière direction et messages. **HypeAgent** regroupe Doge/Shiba ; **ViralAgent** regroupe Pepe/Bonk : ces classes abstraites matérialisent les deux alliances. **Master** hérite aussi d'Agent, mais reste immobile.

Polymorphisme : quatre comportements

Le moteur appelle **move(Simulation)** sur chaque agent actif. Doge choisit une direction et 1 à 3 cases ; Shiba une diagonale et 1 à 2 cases ; Pepe une direction et 1 case ; Bonk une direction et 1 à 2 cases. **travel()** gère collisions, énergie et livraison. La redéfinition de **Master.move()** est vide ; les Maîtres sont exclus de la liste mobile.

Encapsulation : protéger les responsabilités

La grille d'obstacles et les collections du moteur sont privées. **agents()** et **events()** exposent des listes non modifiables. Les classes du modèle restent internes au package ; leur état est partagé entre moteur et rendu. Cette encapsulation est partielle : une vue non modifiable ne rend pas chaque Agent immuable. La fenêtre ne décide ni des combats ni des scores.

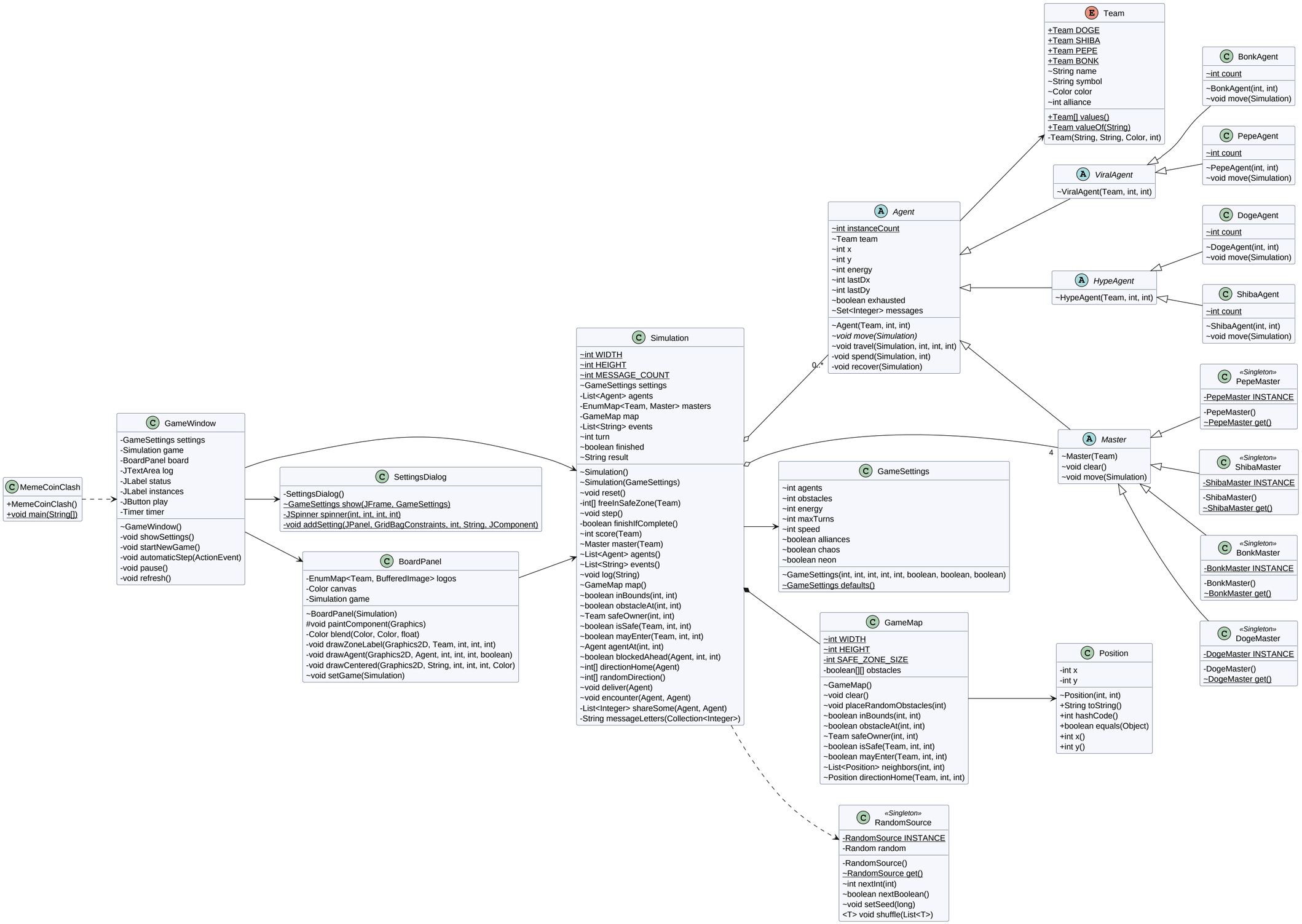
Singleton : quatre Maîtres uniques

DogeMaster, **ShibaMaster**, **PepeMaster** et **BonkMaster** possèdent chacun une instance statique, un constructeur privé et **get()**. Leur état est réinitialisé pour une nouvelle partie. Le programme gère une partie active ; ces instances sont partagées au niveau du processus. **RandomSource** centralise tous les tirages. Les compteurs statiques mesurent les instances créées depuis le lancement.

Diagramme UML de classes complet

Attributs, méthodes, constructeurs, visibilités et relations du programme

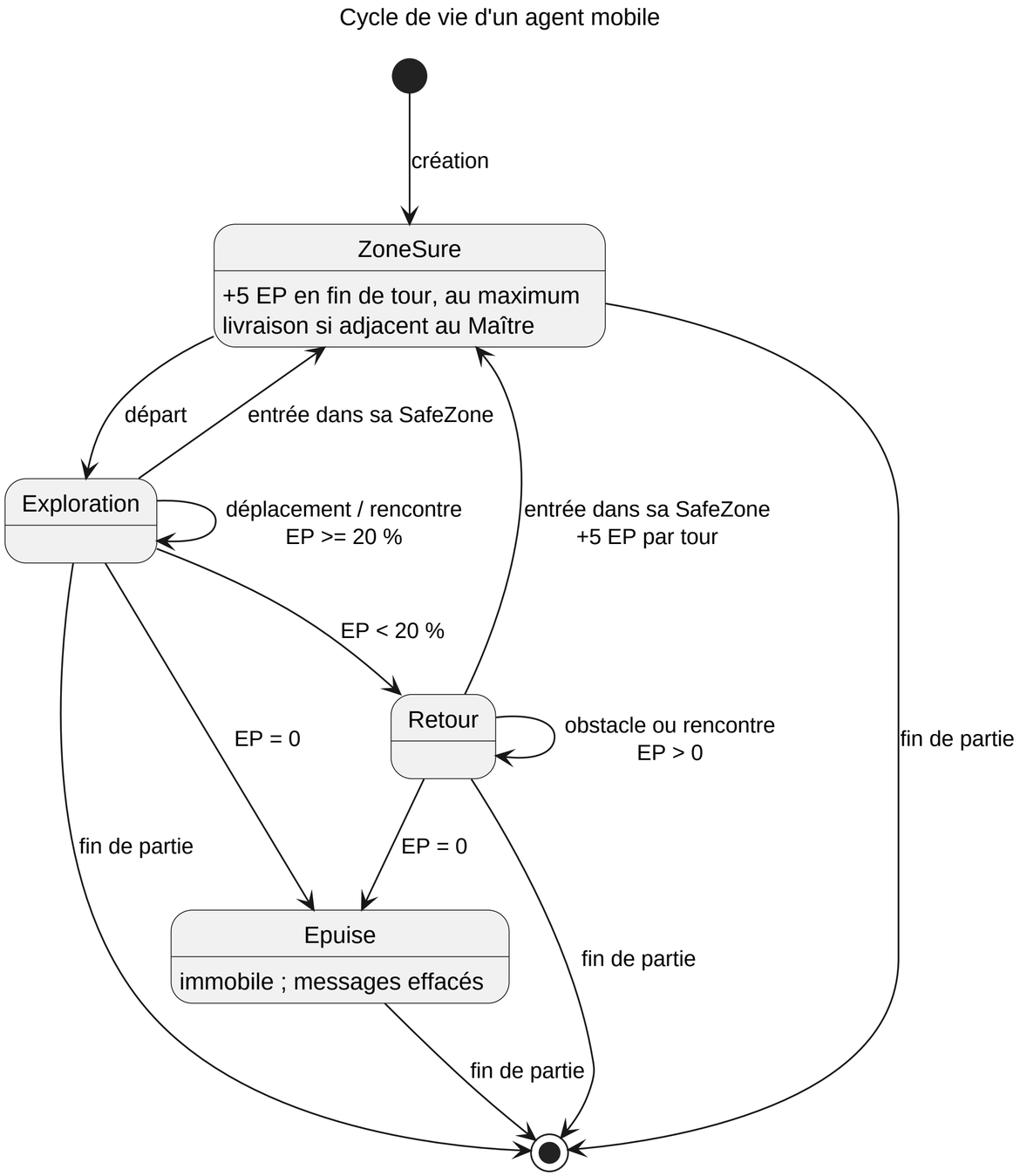
MemeCoin Clash — diagramme de classes complet (déclarations du code)

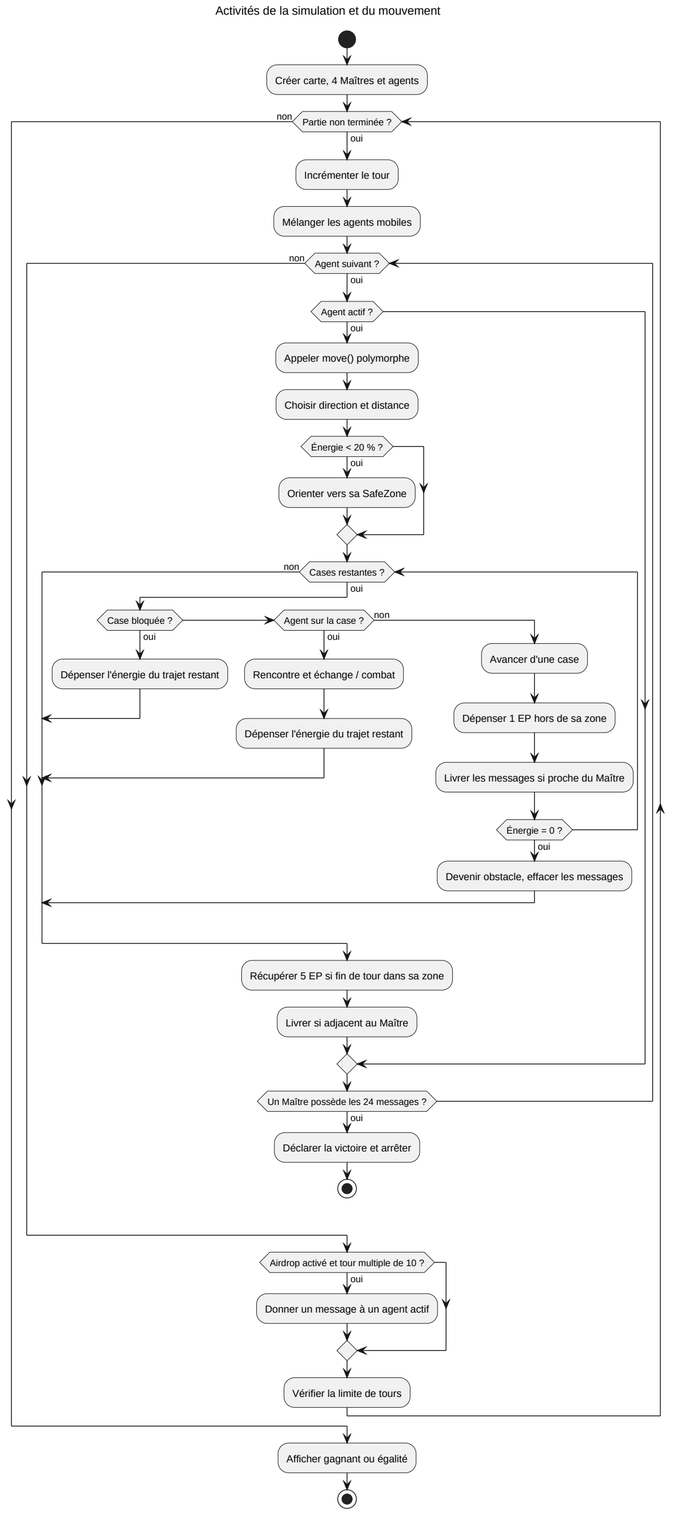


+ public ; - privé ; # protégé ; ~ package. Souligné : statique ; italique : abstrait. Les méthodes héritées ne sont pas répétées. Les membres synthétiques du compilateur sont exclus.

Diagrammes de comportement

États des agents mobiles et activités de la simulation, incluant les deux boucles





EP : points d'énergie. La récupération et la livraison ne réactivent pas un agent épuisé. La fin de partie termine chaque état.

Règles et déroulement d’une partie

Carte, déplacements, interactions, énergie et victoire

Carte et initialisation

L’arène comprend **20 × 14 cases**. Les quatre zones sûres de 3 × 3 occupent 36 cases sur 280, soit 12,9 % de la surface. Chaque zone est réservée à sa faction. Une case contient au plus un occupant. De 2 à 8 agents par faction commencent dans leur zone ; les messages A à X sont répartis entre eux sans disparition, quelle que soit cette population.

Un tour discret et séquentiel

La liste mobile est mélangée à chaque tour. Chaque agent actif agit une fois et ses effets s’appliquent immédiatement. Le minuteur Swing déclenche les tours à intervalle configuré (450 ms par défaut) ; Pause et Un tour permettent l’observation. Maîtres et agents épuisés ne bougent pas. La dernière direction permet d’éviter un blocage répété.

Énergie : une dépense calibrée

L’énergie initiale vaut 24 par défaut (réglable de 10 à 50). Hors de sa zone, chaque case parcourue coûte 1 EP. Une collision consomme le coût des cases restant à parcourir. Sous 20 % de l’énergie maximale, le déplacement est réduit à une case orientée vers le camp. Une fin de tour dans sa zone rend 5 EP, sans dépasser le maximum. À 0 EP, l’agent devient un obstacle permanent et ses messages sont effacés.

Rencontres : trois traitements

Une rencontre survient lorsqu’une case occupée appartient au trajet prévu ; les agents restent sur deux cases adjacentes, diagonales comprises. **Même faction :** union des messages pour les deux agents. **Même alliance :** copie aléatoire de 1 ou 2 messages inédits dans chaque sens, selon disponibilité. **Factions adverses :** gagnant aléatoire, transfert de 1 ou 2 messages inédits retirés au perdant. Les ensembles éliminent les doublons ; le journal affiche les lettres transmises.

Livraison, score et fin

Un agent adjacent à son Maître lui copie ses messages. Seuls les messages distincts du Maître comptent. Après chaque action, une collection complète arrête la partie. Sinon, la limite de tours (250 par défaut) désigne le meilleur score ou une égalité. Les réglages proposent aussi la désactivation des alliances, un airdrop tous les 10 tours et un thème néon. L’airdrop utilise uniquement A à X et ne crée aucune lettre supplémentaire.

Validation et organisation

Exécution, conformité et contributions du groupe

Validation du moteur

La suite **SimulationTest** réussit après compilation en cible Java 17. Elle vérifie voisinage 3/5/8, quatre Singletons, populations 2/5/8, 24 messages initiaux, 70 obstacles et limite de tours. À chaque tour : coordonnées, occupation unique, énergie et zones réservées. Des scénarios ciblent union, alliances, transfert adverse, collision, retour, récupération, livraison, victoire, égalité et réinitialisation. Deux exécutions avec la même graine reproduisent le même état. Le plateau est rendu dans une image ; les commandes de fenêtre restent à vérifier en session graphique interactive.

Lancer et vérifier

Exécutable : `java -jar output/app/MemeCoinClash.jar` (Java 17 ou plus, logos inclus). **Windows :** `run.cmd` et `test.cmd`. **Unix :** `sh run.sh` et `sh test.sh`. Compiler les sources nécessite le JDK ; jouer avec le JAR ne nécessite aucune bibliothèque Java externe.

Attentes couvertes et adaptations

Le sujet autorise une adaptation de l'univers et des règles (p. 1). Le projet conserve quatre populations, deux alliances, pas discrets, héritage, encapsulation, polymorphisme, compteurs statiques, quatre Maîtres Singletons, hasard centralisé et représentation graphique. Les trois diagrammes figurent aux pages 3 et 4. Les identifiants entiers A à X remplacent les chaînes de messages ; leurs ensembles assurent l'unicité.

Répartition anonyme des rôles

Informations attendues : contributions réelles des membres. Les rôles A, B, C et éventuellement D seront décrits anonymement après confirmation, sans attribuer de travaux par supposition.

Limites et références

Le retour utilise une direction simple ; le plus court chemin est un bonus (sujet, p. 6). Météorites, virus, coordination et statistiques multi-parties ne sont pas implémentés. Référence : A MAS Polymorphic Simulation, Concept Objet, M2 ISEN Lille, attentes p. 5-6, remise p. 7. Logos : cryptologos.cc (Doge, Shiba, Pepe) et brand.bonkcoin.com (Bonk).